

Tribeca

Audit



Presented by:

OtterSec contact@osec.io

Robert Chen notdeghost@osec.io

Mohan Pedhapati s1r1us@osec.io

Kevin Chow kchow@osec.io

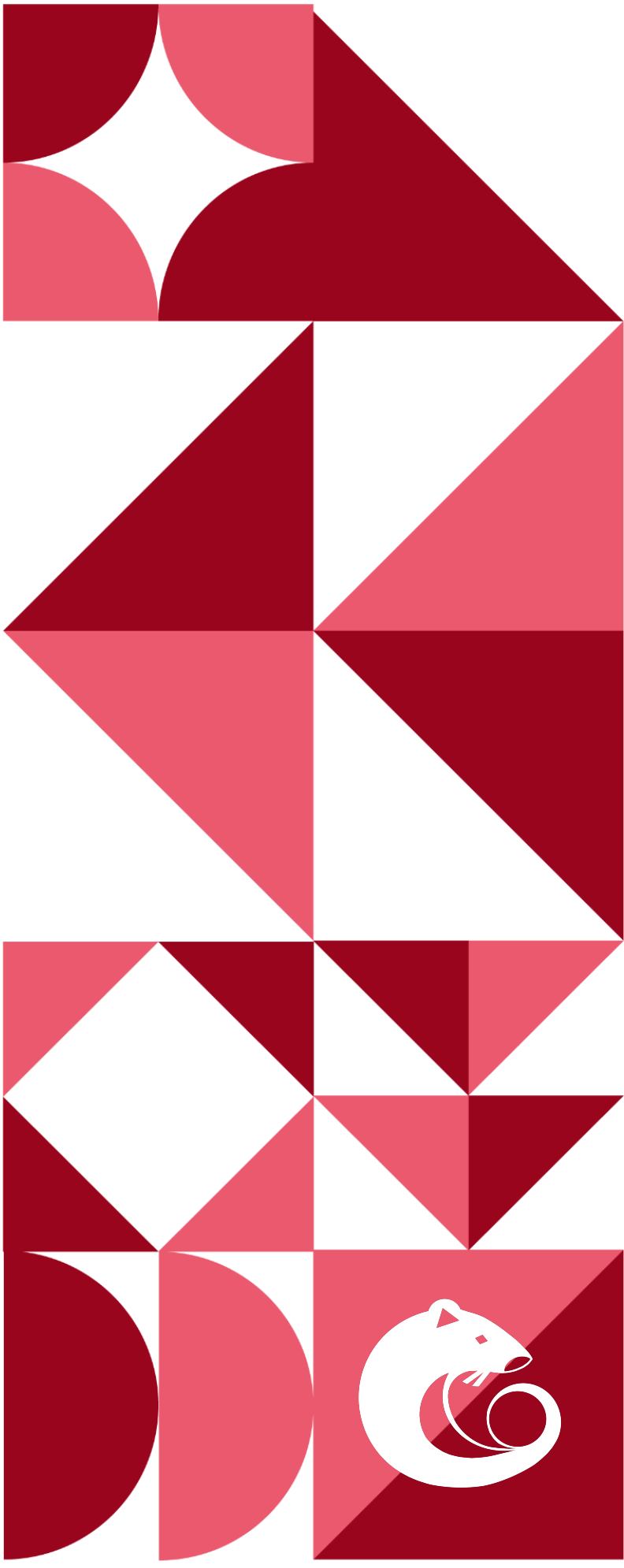


Table of Contents

Table of Contents **1**

01 | Executive Summary **2**
Overview 2
Key Findings 2

02 | Scope **3**

03 | Procedure **4**

04 | Findings **5**
Proof of Concepts 5

05 | Vulnerabilities **7**
OS-TRB-ADV-00 [High]: Locker Whitelist Bypass 8
OS-TRB-ADV-01 [High] : Insufficient checks for lock privileges revoking 10

06 | General Findings **12**
OS-TRB-SUG-00: Restrict timelock delay 13
OS-TRB-SUG-01: Missing Vote Struct closing 14

07 | Appendix **15**
Appendix A: Program Files 15
Appendix B: Implementation Security Checklist 17
Unsafe arithmetic 17
Account security 17
Input Validation 17
Miscellaneous 18
Appendix C: Proof of Concepts 19
Appendix D: Vulnerability Rating Scale 20

© 2022 OtterSec LLC. All Rights Reserved.

01 | Executive Summary

Overview

Ship Capital engaged OtterSec to perform an assessment of the `govern` and `locked-voter` programs prior to deployment.

This assessment was conducted between May 9th and May 25th.

Critical vulnerabilities were communicated to the team prior to the delivery of the report to speed up remediation. After delivering our audit report, we worked closely with the team to streamline patches and confirm remediation.

Key Findings

The following is a summary of the major findings in this audit.

- 4 findings total
- 2 vulnerabilities which could bypass program functionality

As part of this audit, we also provided proof of concepts for each vulnerability to prove exploitability and enable simple regression testing. These scripts can be found at https://osec.io/pocs/tribeca. For a full list, see Appendix C.

© 2022 OtterSec LLC. All Rights Reserved.

02 | Scope

We received the program from Ship Capital and began the audit on May 10th, 2022. The source code was delivered to us in a git repository at https://github.com/TribecaHQ/tribeca. This audit was performed against commit 194e395.

There were a total of two programs included in this audit. A brief description of the two programs is as follows. A full list of program files and hashes can be found in Appendix A.

Name	Description
`govern`	Responsible for voting, proposals, and queuing transactions.
`locked-voter`	Locks up user provided governance tokens and determines voting power.

© 2022 OtterSec LLC. All Rights Reserved.

03 | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an onchain program. In other words, there is no way to steal tokens or deny service, ignoring any Solana specific quirks such as account ownership issues. An example of a design vulnerability would be an onchain oracle which could be manipulated by flash loans or large deposits.

On the other hand, auditing the implementation of the program requires a deep understanding of Solana's execution model. Some common implementation vulnerabilities include account ownership issues, arithmetic overflows, and rounding bugs. For a non-exhaustive list of security issues we check for, see Appendix B. Two such issues that were identified here included OS-TRB-ADV-00 and OS-TRB-ADV-01, which involved inaccurate account checks.

Implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program. For example, here we found an mismatch between checks of `govern` and `goki` (OS-TRB-SUG-00).

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach any target in a team of two. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

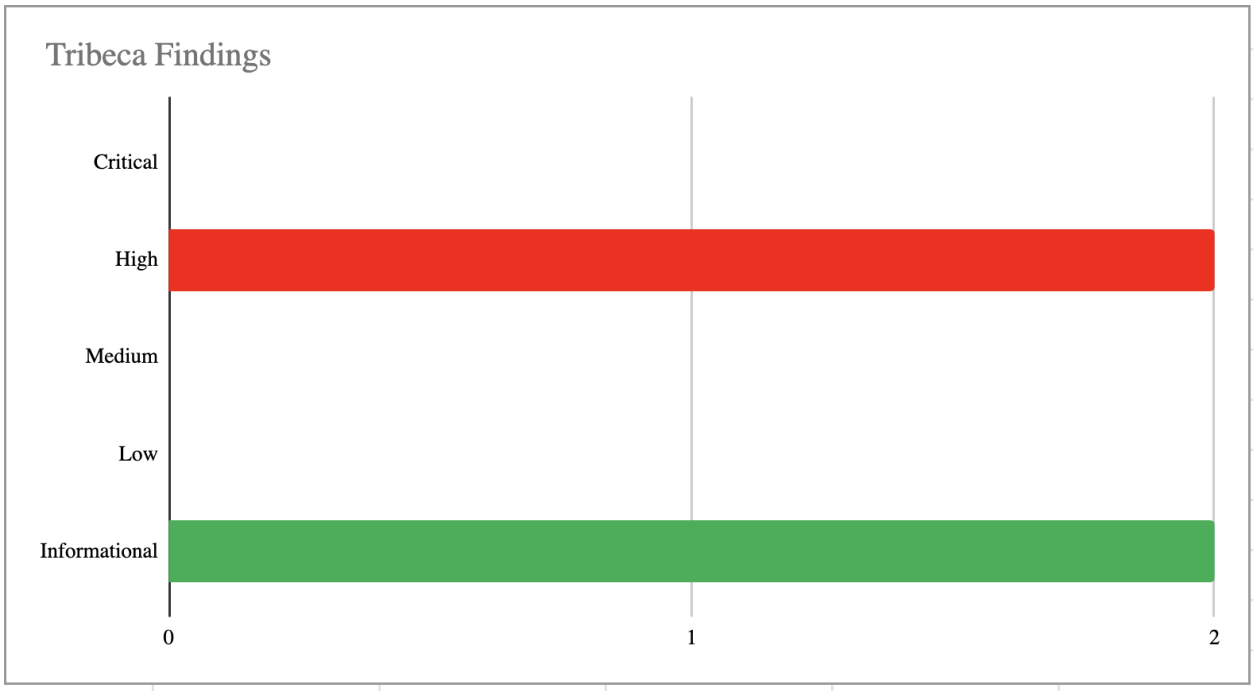
© 2022 OtterSec LLC. All Rights Reserved.

04 | Findings

Overall, we report 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings don't have an immediate impact but will help mitigate future vulnerabilities.

The below chart displays the findings by severity.



Proof of Concepts

For each vulnerability we created a proof of concept to enable easy regression testing. We recommend integrating these as part of a comprehensive test suite. The proof of concept directory structure can be found in Appendix C.

These proof of concepts can be found at https://osec.io/pocs/tribeca.

© 2022 OtterSec LLC. All Rights Reserved.

To run a POC:

```
./run.sh <directory name>
```

For example,

```
./run.sh os-trb-adv-00
```

Each proof of concept comes with its own patch file which modifies the existing test framework to demonstrate the relevant vulnerability. We also recommend integrating these patches into the test suite to prevent regressions.

© 2022 OtterSec LLC. All Rights Reserved.

05 | Vulnerabilities

Here we present a technical analysis of the vulnerabilities we identified during our audit of the `govern` and `locked-voter` programs. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

Rating criterion can be found in Appendix D.

ID	Severity	Status	Description
OS-TRB-ADV-00	**High**	**TODO**	Insufficient account checks on locker during calls
OS-TRB-ADV-01	**High**	**TODO**	Insufficient account checks on locker account during revocation

© 2022 OtterSec LLC. All Rights Reserved.

OS-TRB-ADV-00 [High]: Locker Whitelist Bypass

Description

The locked-voter program supports a privileged locking mechanism using `ApproveProgramLockPrivilege` instruction where only whitelisted programs and escrow owners can lock the tokens in the locker. This instruction does not check the governor of the locker and the provided governor account is the same. This results in creating a whitelist entry for any locker using a fake governor account which effectively bypasses the whitelist mechanism.

programs/locked-voter/src/instructions/approve_program_lock_privilege.rs

```
#[derive(Accounts)]
pub struct ApproveProgramLockPrivilege<'info> {
    /// The [Locker].
    pub locker: Account<'info, Locker>,
    /// [LockerWhitelistEntry].
    [...]
    /// Governor for the [Locker].
    pub governor: Account<'info, Governor>,

    /// Smart wallet on the [Governor].
    pub smart_wallet: Signer<'info>,
    [...]
}
[...]
impl<'info> Validate<'info> for ApproveProgramLockPrivilege<'info> {
    fn validate(&self) -> Result<()> {
        assert_keys_eq!(self.governor.smart_wallet,
self.smart_wallet); [0]
        invariant!(
            self.executable_id.executable,
            "program_id must be an executable"
        );

        Ok(())
    }
}
```

© 2022 OtterSec LLC. All Rights Reserved.

Notice in the above snippet that it [0] only checks if `self.governor.smart_wallet` and the `self.smart_wallet` are same but doesn't check if the `self.locker.governor` and `self.governor` are same.

The same issue exists in the `check_whitelisted` function of `lock` instruction.

Proof of Concept

This exploit allows an attacker to lock the tokens in the privileged lockers. Consider the following scenario.

1. Assume there is a privileged locker L where only program P and escrow owner E can lock.
2. Create a smart wallet and a governor named fakeG using the created smart wallet.
3. Create a fake whitelist entry by sending a `approve_program_lock_privilege` transaction with accounts privileged locker L, and fake governor fakeG instead of actual locker governor.
4. Using the above created fake whitelisted entry, tokens can be locked in the locker L without the need of program P and escrow E.

Remediation

The program should check if the `self.locker.governor == self.governor` while creating a whitelist entry.

© 2022 OtterSec LLC. All Rights Reserved.

OS-TRB-ADV-01 [High] : Insufficient checks for lock privileges revoking

Description

It was found that anyone can revoke the program lock privileges. This was due to the fact that the `RevokeProgramLockPrivilege` does not check if the governor of the locker and the provided governor are the same.

The following snippet shows the affected code. It can be noticed [0] that there is no check if the `self.locker.governor` and `self.governor` are the same.

programs/locked-voter/src/instructions/revoke_program_lock_privilege.rs

```
/// Accounts for [locked_voter::revoke_program_lock_privilege].
#[derive(Accounts)]
pub struct RevokeProgramLockPrivilege<'info> {
    /// The [Locker].
    pub locker: Account<'info, Locker>,
    /// [LockerWhitelistEntry].
    #[account(mut, has_one = locker, close = payer)]
    pub whitelist_entry: Account<'info, LockerWhitelistEntry>,

    /// Governor for the [Locker].
    pub governor: Account<'info, Governor>,
    /// Smart wallet on the [Governor].
    pub smart_wallet: Signer<'info>,
    /// Payer of the initialization.
    #[account(mut)]
    pub payer: Signer<'info>,
}
[...]
impl<'info> Validate<'info> for RevokeProgramLockPrivilege<'info> {
    fn validate(&self) -> Result<()> {
        assert_keys_eq!(self.governor.smart_wallet,
self.smart_wallet); [0]
        assert_keys_eq!(self.whitelist_entry.locker, self.locker);
        Ok(())
    }
}
```

© 2022 OtterSec LLC. All Rights Reserved.

Proof of Concept

Consider the following scenario

1. Assume there is a privileged locker L where only program P and escrow owner E can lock.
2. Create a smart wallet and a governor named fakeG using the created smart wallet.
3. Revoke any whitelist entry by sending a `revoke_program_lock_privilege` transaction with accounts privileged locker L, and fake governor fakeG instead of actual locker governor.
4. Notice that the whitelist entry can be revoked.

Remediation

It is recommended to check if `self.locker.governor == self.governor` while creating a whitelist entry similar to OS-TRB-ADV-00.

© 2022 OtterSec LLC. All Rights Reserved.

06 | General Findings

Here we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they do represent antipatterns and could introduce a vulnerability in the future.

ID	Description
OS-TRB-SUG-00	Time lock delay doesn't have an upper bound check.
OS-TRB-SUG-01	No way to close vote struct.

© 2022 OtterSec LLC. All Rights Reserved.

OS-TRB-SUG-00: Restrict timelock delay

Description

The `timelock_delay_seconds` governor parameter is missing the maximum delay check. As the `create_transaction_with_timelock` of goki smart wallet checks if the delay is below `goki::MAX_DELAY_SECONDS`, it is good to validate `timelock_delay_seconds` to be below `MAX_DELAY_SECONDS` for better user experience.

programs/govern/src/lib.rs

```
#[access_control(ctx.accounts.validate())]
pub fn set_governance_params(
    ctx: Context<SetGovernanceParams>,
    params: GovernanceParameters,
) -> Result<()> {
    let prev_params = ctx.accounts.governor.params;
    ctx.accounts.governor.params = params;

    emit!(GovernorSetParamsEvent {
        governor: ctx.accounts.governor.key(),
        prev_params,
        params,
    });

    Ok(())
}
```

Remediation

It is recommended to validate if `timelock_delay_seconds` is below `goki::MAX_DELAY_SECONDS`.

© 2022 OtterSec LLC. All Rights Reserved.

OS-TRB-SUG-01: Missing Vote Struct closing

Description

It was noticed that currently there is no way to close the vote struct to reclaim the lamports even after the voting ends.

This imposes a cost on voting against a proposal which disincentivies voting.

Remediation

We would recommend providing a way to close vote struct to reclaim the lamports provided by the payer. It is ideal to provide a way to close the account structures if they are not useful anymore.

© 2022 OtterSec LLC. All Rights Reserved.

07 | Appendix

Appendix A: Program Files

Below are the files in scope for this audit and their corresponding truncated sha256 hashes.

```
whitelist-tester
  Cargo.toml                                          9ff1b6715188eeb7bed914df3465fc01
  README.md                                           e38f9afdde49749e6ee1b5adfabeb58d
  Xargo.toml                                          815f2dfb6197712a703a8e1f75b03c69
  src/lib.rs                                          d40e74fe6d5af55042069621b98bc834
simple-voter
  Cargo.toml                                          4049e68ddb2c5aa8c8e3983042573477
  README.md                                           a1190fc7fa6d0b81421844047ac1c216
  Xargo.toml                                          815f2dfb6197712a703a8e1f75b03c69
  src/account_validators.rs                           6570f5663696b36b3dd1be2fd338ad74
  src/lib.rs                                          4f6a4d68ac196ca7e8464d6aa23615be
  src/processor/voter.rs                              6b6f177831c0daf129306fbb605f9e1a
  src/processor/mod.rs                                248ae7defb42463a0bcf1ff4f01d60bf
  src/processor/proposer.rs                           fcb1947181799ea24185ba66af177481
  src/state.rs                                        384d5f035c375bfdf5c6296527a3a07e
  src/token_cpi.rs                                    30ee17af639702d26ae808d91bef976d
  src/macros.rs                                       01a6ddc094e2bbc73c66e05f5905cf35
govern
  Cargo.toml                                          e62fd7c9aba5ba28743c22abf789cd52
  README.md                                           cf7b47e895eb1a5ebbf03f9fa62d0053
  Xargo.toml                                          815f2dfb6197712a703a8e1f75b03c69
  src/proposal.rs                                     c2d3defad415f8738a4cc39ba76b6907
  src/account_validators.rs                           74963ae28655b11dc8166c3d8590a60a
  src/events.rs                                       86021c4f70bc8b07541c060986a31984
  src/lib.rs                                          72830c67cf260b23741813ed47713da7
  src/account_structs.rs                              4f77d66010f8827dbee543d3f55e3c55
  src/state.rs                                        729e856b17d620fde89af515f1e6b819
  src/macros.rs                                       9e40c167af21a4d306736d231b8c27b3
locked-voter
  Cargo.toml                                          8307769a33610a6655aedfe634cc68c8
  README.md                                           4aacd07d0385891f5d2ef0079db5a64a
  Xargo.toml                                          815f2dfb6197712a703a8e1f75b03c69
  src/instructions/new_locker.rs                      3dd9e5c71cf9def040acd9ad97fe4c6f
  src/instructions/lock_with_whitelist_entry.rs       6db5023d3e77b4d3b29e6855c343620a
  src/instructions/approve_program_lock_privilege.rs  35f0d3095b389fd075f1154c60a7abdd
  src/instructions/set_locker_params.rs               6202dbbf82714b3a4b8e7076962b6c6c
  src/instructions/lock_permissionless.rs             d2315af4c8c4a9566e11bec6aed48687
  src/instructions/set_vote_delegate.rs               913d5baf941fc613c86c7a51828341cb
  src/instructions/lock_with_whitelist.rs             44828dd66208357c7fcebabc0d68e488
  src/instructions/new_escrow.rs                      737374346572b214cfa40db4146ff588
  src/instructions/mod.rs                             2731f4d92247a9a3fc639ff869c18fc2
  src/instructions/revoke_program_lock_privilege.rs   0f18d9a885592ae3b6838d34ab57f05b
  src/instructions/exit.rs                            e6aa2eaff7897c197e7bd3af80be8bc1
  src/instructions/lock.rs                            85eb38fa087bdeca30e6e70f1eb4f894
  src/instructions/activate_proposal.rs               15dba2fd8ede13549044e2474cc66ef4
  src/instructions/cast_vote.rs                       144d425d5d642242cd7b651c1c4c3418
```

© 2022 OtterSec LLC. All Rights Reserved.

src/locker.rs	6a3eb713b9bdbad143381789a051fb19
src/lib.rs	261335603a46a0e7fafb2344a137f076
src/state.rs	623a2750f7a834acd0ec72d61205a40f
src/macros.rs	ece00ad8e52c683fb00e9544893b8322

© 2022 OtterSec LLC. All Rights Reserved.

Appendix B: Implementation Security Checklist

Unsafe arithmetic

Integer underflows or overflows	Unconstrained input sizes could lead to integer over or underflows, causing potentially unexpected behavior. Ensure that for unchecked arithmetic, all integers are properly bounded.
Rounding	Rounding should always be done against the user to avoid potentially exploitable off-by-one vulnerabilities.
Conversions	Rust `as` conversions can cause truncation if the source value does not fit into the destination type. While this is not undefined behavior, such truncation could still lead to unexpected behavior by the program.

Account security

Account Ownership	Account ownership should be properly checked to avoid type confusion attacks. For Anchor, the safety of unchecked accounts should be clearly justified and immediately obvious.
Accounts	For non-Anchor programs, the type of the account should be explicitly validated to avoid type confusion attacks.
Signer Checks	Privileged operations should ensure that the operation is signed by the correct accounts.
PDA Seeds	PDA seeds are uniquely chosen to differentiate between different object classes, avoiding collision.

Input Validation

Timestamps	Timestamp inputs should be properly validated against the current clock time. Timestamps which are meant to be in the future should be explicitly validated so.
Numbers	Sane limits should be put on numerical input data to mitigate the risk of unexpected over and underflows. Input data should be constrained to the smallest size type possible, and upcasted for unchecked arithmetic.
Strings	Strings should have sane size restrictions to prevent denial of service conditions

© 2022 OtterSec LLC. All Rights Reserved.

Internal State	If there is internal state, ensure that there is explicit validation on the input account's state before engaging in any state transitions. For example, only open accounts should be eligible for closing.

Miscellaneous

Libraries	Out of date libraries should not include any publicly disclosed vulnerabilities
Clippy	`cargo clippy` is an effective linter to detect potential anti-practices.

© 2022 OtterSec LLC. All Rights Reserved.

Appendix C: Proof of Concepts

Below are the provided proof of concept files and their truncated sha256 hashes.

```
os-trb-adv-00
  run.sh              2b542f334f746771a38b8a7f8fc18262
  patch               be5228107778883905a6ed87ad8fd789
os-trb-adv-0
  run.sh              2b542f334f746771a38b8a7f8fc18262
  patch               9734b115188789a34cc5aaa40bc1a220
run.sh                18532da56315cbea696c2c4ff5344bbc
```

© 2022 OtterSec LLC. All Rights Reserved.

Appendix D: Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities which **immediately** lead to loss of user funds with minimal preconditions Examples: - Misconfigured authority/token account validation - Rounding errors on token transfers
High	Vulnerabilities which **could** lead to loss of user funds but are potentially difficult to exploit. Examples: - Loss of funds requiring specific victim interactions - Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities which could lead to denial of service scenarios or degraded usability. Examples: - Malicious input cause computation limit exhaustion - Forced exceptions preventing normal use
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. Examples: - Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as *general findings*. Examples: - Explicit assertion of critical internal invariants - Improved input validation - Uncaught Rust errors (vector out of bounds indexing)

© 2022 OtterSec LLC. All Rights Reserved.